

grumpiness score.¹³ If the mean is 17 and the standard deviation is 5 then my standardised grumpiness score would be¹⁴

$$z = \frac{35 - 17}{5} = 3.6$$

To interpret this value, recall the rough heuristic that I provided in Section 5.2.5, in which I noted that 99.7% of values are expected to lie within 3 standard deviations of the mean. So the fact that my grumpiness corresponds to a z score of 3.6 indicates that I'm very grumpy indeed. Later on, in Section 9.5, I'll introduce a function called `pnorm()` that allows us to be a bit more precise than this. Specifically, it allows us to calculate a theoretical percentile rank for my grumpiness, as follows:

```
> pnorm( 3.6 )  
[1] 0.9998409
```

At this stage, this command doesn't make too much sense, but don't worry too much about it. It's not important for now. But the output is fairly straightforward: it suggests that I'm grumpier than 99.98% of people. Sounds about right.

In addition to allowing you to interpret a raw score in relation to a larger population (and thereby allowing you to make sense of variables that lie on arbitrary scales), standard scores serve a second useful function. Standard scores can be compared to one another in situations where the raw scores can't. Suppose, for instance, my friend also had another questionnaire that measured extraversion using a 24 items questionnaire. The overall mean for this measure turns out to be 13 with standard deviation 4; and I scored a 2. As you can imagine, it doesn't make a lot of sense to try to compare my raw score of 2 on the extraversion questionnaire to my raw score of 35 on the grumpiness questionnaire. The raw scores for the two variables are "about" fundamentally different things, so this would be like comparing apples to oranges.

What about the standard scores? Well, this is a little different. If we calculate the standard scores, we get $z = (35 - 17)/5 = 3.6$ for grumpiness and $z = (2 - 13)/4 = -2.75$ for extraversion. These two numbers *can* be compared to each other.¹⁵ I'm much less extraverted than most people ($z = -2.75$) and much grumpier than most people ($z = 3.6$): but the extent of my unusualness is much more extreme for grumpiness (since 3.6 is a bigger number than 2.75). Because each standardised score is a statement about where an observation falls *relative to its own population*, it is possible to compare standardised scores across completely different variables.

5.7

Correlations

Up to this point we have focused entirely on how to construct descriptive statistics for a single variable. What we haven't done is talked about how to describe the relationships *between* variables in the data. To do that, we want to talk mostly about the **correlation** between variables. But first, we need some data.

¹³I haven't discussed how to compute z -scores, explicitly, but you can probably guess. For a variable `x`, the simplest way is to use a command like `(x - mean(x)) / sd(x)`. There's also a fancier function called `scale()` that you can use, but it relies on somewhat more complicated R concepts that I haven't explained yet.

¹⁴Technically, because I'm calculating means and standard deviations from a sample of data, but want to talk about my grumpiness relative to a population, what I'm actually doing is *estimating* a z score. However, since we haven't talked about estimation yet (see Chapter 10) I think it's best to ignore this subtlety, especially as it makes very little difference to our calculations.

¹⁵Though some caution is usually warranted. It's not always the case that one standard deviation on variable A corresponds to the same "kind" of thing as one standard deviation on variable B. Use common sense when trying to determine whether or not the z scores of two variables can be meaningfully compared.

Table 5.1: Descriptive statistics for the parenthood data.

variable	min	max	mean	median	std. dev	IQR
Dan's grumpiness	41	91	63.71	62	10.05	14
Dan's hours slept	4.84	9.00	6.97	7.03	1.02	1.45
Dan's son's hours slept	3.25	12.07	8.05	7.95	2.07	3.21

5.7.1 The data

After spending so much time looking at the AFL data, I'm starting to get bored with sports. Instead, let's turn to a topic close to every parent's heart: sleep. The following data set is fictitious, but based on real events. Suppose I'm curious to find out how much my infant son's sleeping habits affect my mood. Let's say that I can rate my grumpiness very precisely, on a scale from 0 (not at all grumpy) to 100 (grumpy as a very, very grumpy old man). And, let's also assume that I've been measuring my grumpiness, my sleeping patterns and my son's sleeping patterns for quite some time now. Let's say, for 100 days. And, being a nerd, I've saved the data as a file called `parenthood.Rdata`. If we load the data...

```
> load( "parenthood.Rdata" )
> who(TRUE)
-- Name --      -- Class --      -- Size --
parenthood      data.frame      100 x 4
$dan.sleep      numeric      100
$baby.sleep     numeric      100
$dan.grump      numeric      100
$day            integer      100
```

... we see that the file contains a single data frame called `parenthood`, which contains four variables `dan.sleep`, `baby.sleep`, `dan.grump` and `day`. If we print out the data, here's what we get:

```
> parenthood
  dan.sleep baby.sleep dan.grump day
1      7.59    10.18      56    1
2      7.91    11.66      60    2
3      5.14      7.92      82    3
4      7.71      9.61      55    4

BLAH BLAH BLAH

98      5.38      6.97      82   98
99      7.02      6.56      55   99
100     6.45      7.93      74  100
```

Next, I'll calculate some basic descriptive statistics:

```
> describe( parenthood )
      var   n mean   sd median trimmed  mad   min   max
dan.sleep  1 100  6.97  1.02   7.03    7.00  1.09  4.84  9.00
baby.sleep  2 100  8.05  2.07   7.95    8.05  2.33  3.25 12.07
dan.grump   3 100 63.71 10.05  62.00   63.16  9.64 41.00 91.00
day         4 100 50.50 29.01  50.50   50.50 37.06  1.00 100.00
```

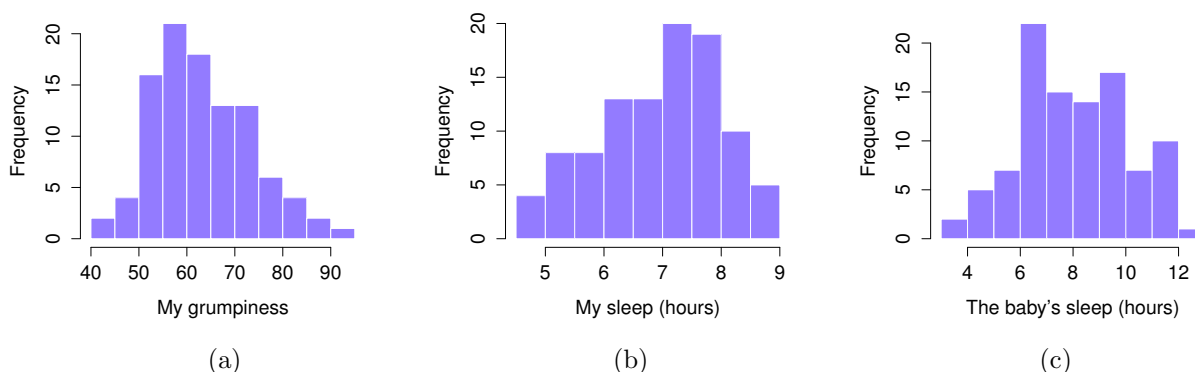


Figure 5.6: Histograms for the three interesting variables in the `parenthood` data set.

	range	skew	kurtosis	se
<code>dan.sleep</code>	4.16	-0.29	-0.65	0.10
<code>baby.sleep</code>	8.82	-0.02	-0.61	0.21
<code>dan.grump</code>	50.00	0.43	-0.04	1.00
<code>day</code>	99.00	0.00	-1.20	2.90

Finally, to give a graphical depiction of what each of the three interesting variables looks like, Figure 5.6 plots histograms. One thing to note: just because R can calculate dozens of different statistics doesn't mean you should report all of them. If I were writing this up for a report, I'd probably pick out those statistics that are of most interest to me (and to my readership), and then put them into a nice, simple table like the one in Table 5.1.¹⁶ Notice that when I put it into a table, I gave everything “human readable” names. This is always good practice. Notice also that I'm not getting enough sleep. This isn't good practice, but other parents tell me that it's standard practice.

5.7.2 The strength and direction of a relationship

We can draw scatterplots to give us a general sense of how closely related two variables are. Ideally though, we might want to say a bit more about it than that. For instance, let's compare the relationship between `dan.sleep` and `dan.grump` (Figure 5.7, left) with that between `baby.sleep` and `dan.grump` (Figure 5.7, right). When looking at these two plots side by side, it's clear that the relationship is *qualitatively* the same in both cases: more sleep equals less grump! However, it's also pretty obvious that the relationship between `dan.sleep` and `dan.grump` is *stronger* than the relationship between `baby.sleep` and `dan.grump`. The plot on the left is “neater” than the one on the right. What it feels like is that if you want to predict what my mood is, it'd help you a little bit to know how many hours my son slept, but it'd be more helpful to know how many hours I slept.

In contrast, let's consider the two scatterplots shown in Figure 5.8. If we compare the scatterplot of “`baby.sleep` v `dan.grump`” (left) to the scatterplot of “`baby.sleep` v `dan.sleep`” (right), the overall strength of the relationship is the same, but the direction is different. That is, if my son sleeps more,

¹⁶Actually, even that table is more than I'd bother with. In practice most people pick *one* measure of central tendency, and *one* measure of variability only.

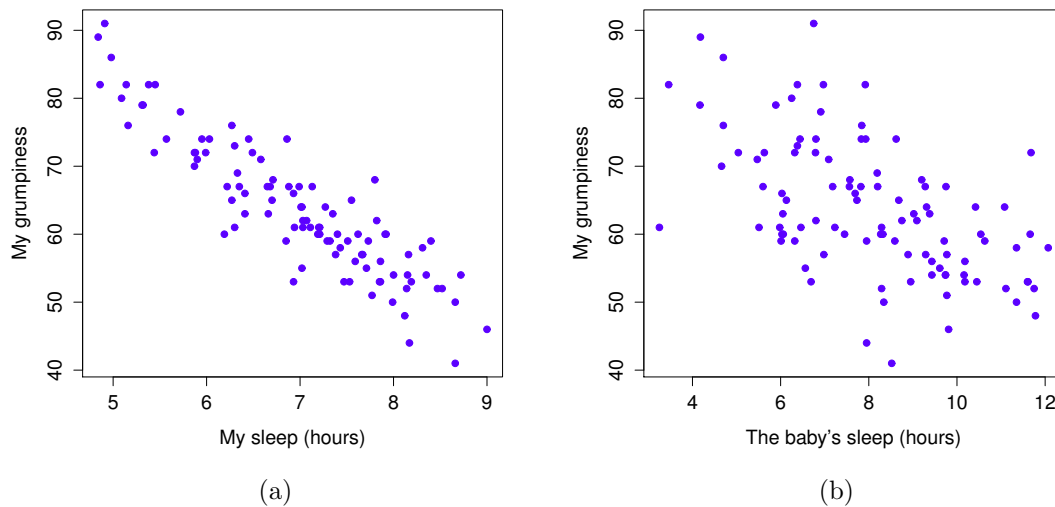


Figure 5.7: Scatterplots showing the relationship between `dan.sleep` and `dan.grump` (left) and the relationship between `baby.sleep` and `dan.grump` (right).

I get *more* sleep (positive relationship, right hand side), but if he sleeps more then I get *less* grumpy (negative relationship, left hand side).

5.7.3 The correlation coefficient

We can make these ideas a bit more explicit by introducing the idea of a **correlation coefficient** (or, more specifically, Pearson's correlation coefficient), which is traditionally denoted by r . The correlation coefficient between two variables X and Y (sometimes denoted r_{XY}), which we'll define more precisely in the next section, is a measure that varies from -1 to 1 . When $r = -1$ it means that we have a perfect negative relationship, and when $r = 1$ it means we have a perfect positive relationship. When $r = 0$, there's no relationship at all. If you look at Figure 5.9, you can see several plots showing what different correlations look like.

The formula for the Pearson's correlation coefficient can be written in several different ways. I think the simplest way to write down the formula is to break it into two steps. Firstly, let's introduce the idea of a **covariance**. The covariance between two variables X and Y is a generalisation of the notion of the variance; it's a mathematically simple way of describing the relationship between two variables that isn't terribly informative to humans:

$$\text{Cov}(X, Y) = \frac{1}{N-1} \sum_{i=1}^N (X_i - \bar{X}) (Y_i - \bar{Y})$$

Because we're multiplying (i.e., taking the “product” of) a quantity that depends on X by a quantity that depends on Y and then averaging¹⁷, you can think of the formula for the covariance as an “average cross product” between X and Y . The covariance has the nice property that, if X and Y are entirely unrelated, then the covariance is exactly zero. If the relationship between them is positive (in the

¹⁷Just like we saw with the variance and the standard deviation, in practice we divide by $N-1$ rather than N .

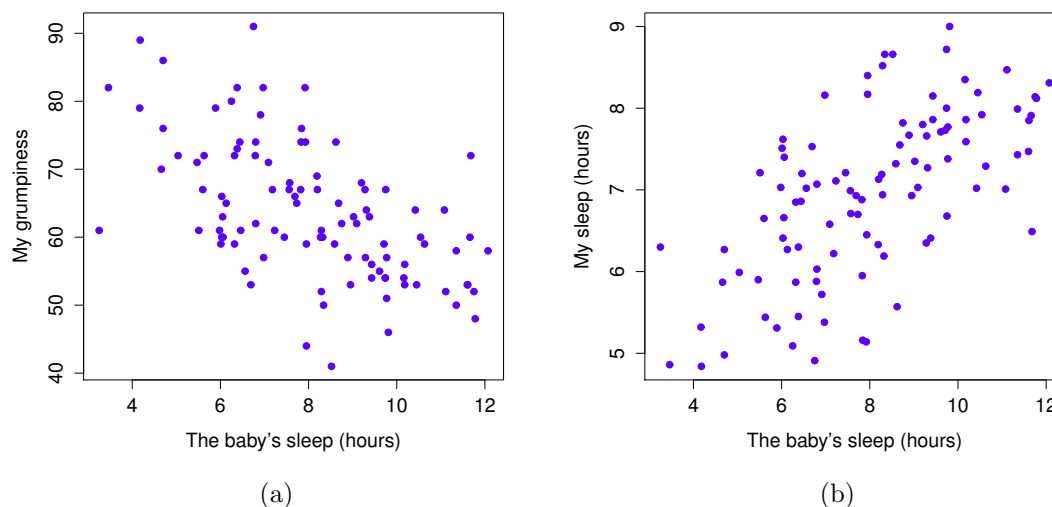


Figure 5.8: Scatterplots showing the relationship between `baby.sleep` and `dan.grump` (left), as compared to the relationship between `baby.sleep` and `dan.sleep` (right).

.....

sense shown in Figure 5.9) then the covariance is also positive; and if the relationship is negative then the covariance is also negative. In other words, the covariance captures the basic qualitative idea of correlation. Unfortunately, the raw magnitude of the covariance isn't easy to interpret: it depends on the units in which X and Y are expressed, and worse yet, the actual units that the covariance itself is expressed in are really weird. For instance, if X refers to the `dan.sleep` variable (units: hours) and Y refers to the `dan.grump` variable (units: grumps), then the units for their covariance are “hours $\times$ grumps”. And I have no freaking idea what that would even mean.

The Pearson correlation coefficient r fixes this interpretation problem by standardising the covariance, in pretty much the exact same way that the z -score standardises a raw score: by dividing by the standard deviation. However, because we have two variables that contribute to the covariance, the standardisation only works if we divide by both standard deviations.¹⁸ In other words, the correlation between X and Y can be written as follows:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{\hat{\sigma}_X \hat{\sigma}_Y}$$

By doing this standardisation, not only do we keep all of the nice properties of the covariance discussed earlier, but the actual values of r are on a meaningful scale: $r = 1$ implies a perfect positive relationship, and $r = -1$ implies a perfect negative relationship. I'll expand a little more on this point later, in Section 5.7.5. But before I do, let's look at how to calculate correlations in R.

5.7.4 Calculating correlations in R

Calculating correlations in R can be done using the `cor()` command. The simplest way to use the command is to specify two input arguments `x` and `y`, each one corresponding to one of the variables. The

¹⁸This is an oversimplification, but it'll do for our purposes.

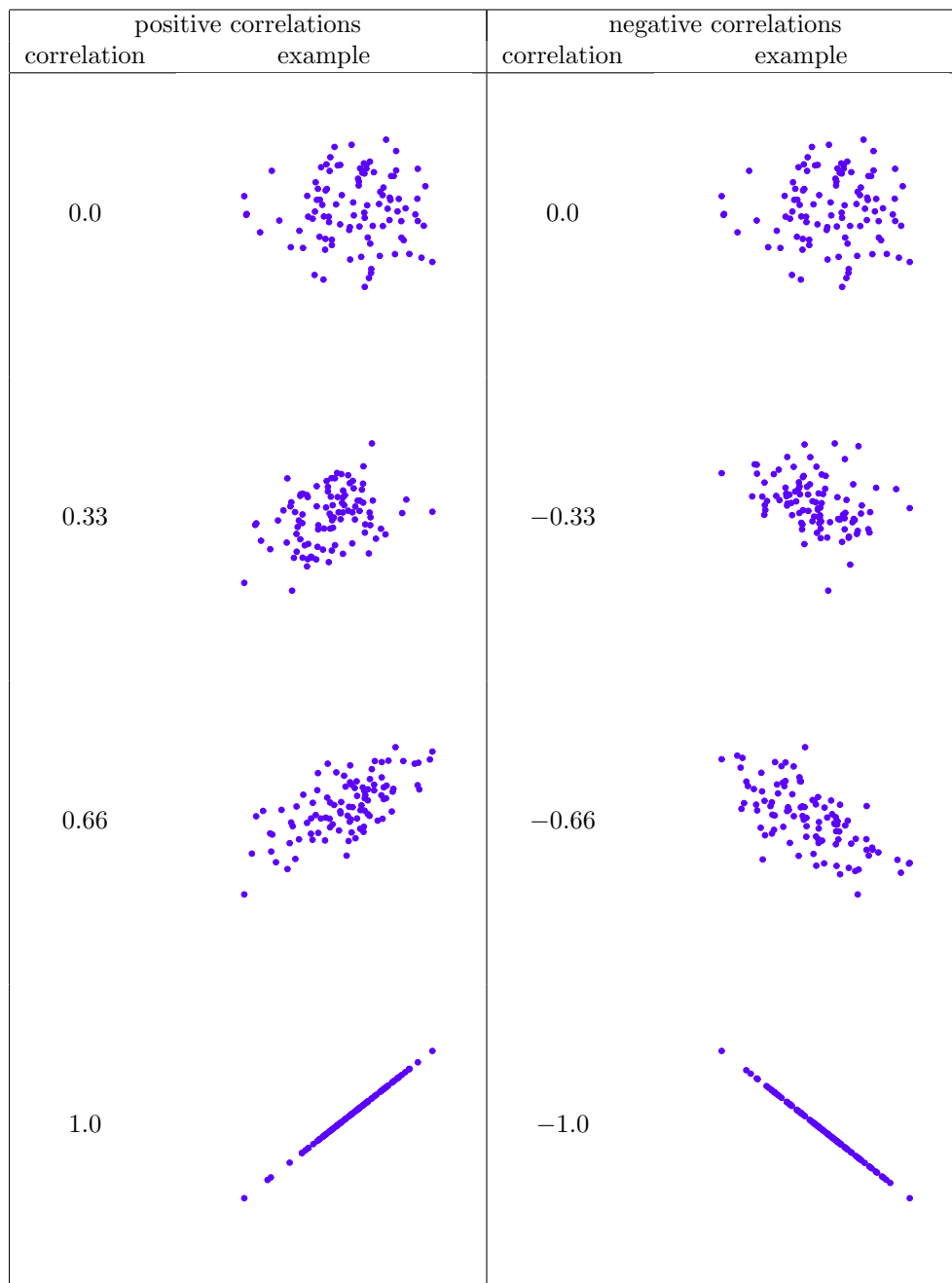


Figure 5.9: Illustration of the effect of varying the strength and direction of a correlation. In the left hand column, the correlations are 0, .33, .66 and 1. In the right hand column, the correlations are 0, -.33, -.66 and -1.

.....

following extract illustrates the basic usage of the function:¹⁹

```
> cor( x = parenthood$dan.sleep, y = parenthood$dan.grump )
[1] -0.903384
```

However, the `cor()` function is a bit more powerful than this simple example suggests. For example, you can also calculate a complete “correlation matrix”, between all pairs of variables in the data frame:²⁰

```
> # correlate all pairs of variables in "parenthood":
> cor( x = parenthood )
      dan.sleep  baby.sleep  dan.grump      day
dan.sleep  1.00000000  0.62794934 -0.90338404 -0.09840768
baby.sleep  0.62794934  1.00000000 -0.56596373 -0.01043394
dan.grump  -0.90338404 -0.56596373  1.00000000  0.07647926
day        -0.09840768 -0.01043394  0.07647926  1.00000000
```

5.7.5 Interpreting a correlation

Naturally, in real life you don’t see many correlations of 1. So how should you interpret a correlation of, say $r = .4$? The honest answer is that it really depends on what you want to use the data for, and on how strong the correlations in your field tend to be. A friend of mine in engineering once argued that any correlation less than .95 is completely useless (I think he was exaggerating, even for engineering). On the other hand there are real cases – even in psychology – where you should really expect correlations that strong. For instance, one of the benchmark data sets used to test theories of how people judge similarities is so clean that any theory that can’t achieve a correlation of at least .9 really isn’t deemed to be successful. However, when looking for (say) elementary correlates of intelligence (e.g., inspection time, response time), if you get a correlation above .3 you’re doing very very well. In short, the interpretation of a correlation depends a lot on the context. That said, the rough guide in Table 5.2 is pretty typical.

However, something that can never be stressed enough is that you should *always* look at the scatterplot before attaching any interpretation to the data. A correlation might not mean what you think it means. The classic illustration of this is “Anscombe’s Quartet” (Anscombe, 1973), which is a collection of four data sets. Each data set has two variables, an X and a Y . For all four data sets the mean value for X is 9 and the mean for Y is 7.5. The, standard deviations for all X variables are almost identical, as are those for the Y variables. And in each case the correlation between X and Y is $r = 0.816$. You can verify this yourself, since I happen to have saved the in a file²¹ called `anscombesquartet.Rdata`. The commands would be:

```
> load( "anscombesquartet.Rdata" )
> cor( X1, Y1 )
[1] 0.8164205
> cor( X2, Y2 )
[1] 0.8162365
```

and so on. You’d think that these four data sets would look pretty similar to one another. They do not. If we draw scatterplots of X against Y for all four variables, as shown in Figure 5.10 we see that all four

¹⁹If you are reading this after having already completed Chapter 11 you might be wondering about hypothesis tests for correlations. R has a function called `cor.test()` that runs a hypothesis test for a single correlation, and the `psych` package contains a version called `corr.test()` that can run tests for every correlation in a correlation matrix; hypothesis tests for correlations are discussed in more detail in Section 15.6.

²⁰An alternative usage of `cor()` is to correlate one set of variables with another subset of variables. If x and y are both data frames with the same number of rows, then `cor(x = X, y = Y)` will produce a correlation matrix that correlates all variables in x with all variables in y .

²¹It’s also stored as a dataset that comes distributed with R. Just type `anscombe` at the command prompt.

Table 5.2: A rough guide to interpreting correlations. Note that I say a *rough* guide. There aren't hard and fast rules for what counts as strong or weak relationships. It depends on the context.

Correlation	Strength	Direction
-1.0 to -0.9	Very strong	Negative
-0.9 to -0.7	Strong	Negative
-0.7 to -0.4	Moderate	Negative
-0.4 to -0.2	Weak	Negative
-0.2 to 0	Negligible	Negative
0 to 0.2	Negligible	Positive
0.2 to 0.4	Weak	Positive
0.4 to 0.7	Moderate	Positive
0.7 to 0.9	Strong	Positive
0.9 to 1.0	Very strong	Positive

of these are *spectacularly* different to each other. The lesson here, which so very many people seem to forget in real life is “*always graph your raw data*”. This will be the focus of Chapter 6.

5.7.6 Spearman's rank correlations

The Pearson correlation coefficient is useful for a lot of things, but it does have shortcomings. One issue in particular stands out: what it actually measures is the strength of the *linear* relationship between two variables. In other words, what it gives you is a measure of the extent to which the data all tend to fall on a single, perfectly straight line. Often, this is a pretty good approximation to what we mean when we say “relationship”, and so the Pearson correlation is a good thing to calculation. Sometimes, it isn't.

One very common situation where the Pearson correlation isn't quite the right thing to use arises when an increase in one variable X really is reflected in an increase in another variable Y , but the nature of the relationship isn't necessarily linear. An example of this might be the relationship between effort and reward when studying for an exam. If you put in zero effort (X) into learning a subject, then you should expect a grade of 0% (Y). However, a little bit of effort will cause a *massive* improvement: just turning up to lectures means that you learn a fair bit, and if you just turn up to classes, and scribble a few things down so your grade might rise to 35%, all without a lot of effort. However, you just don't get the same effect at the other end of the scale. As everyone knows, it takes *a lot* more effort to get a grade of 90% than it takes to get a grade of 55%. What this means is that, if I've got data looking at study effort and grades, there's a pretty good chance that Pearson correlations will be misleading.

To illustrate, consider the data plotted in Figure 5.11, showing the relationship between hours worked and grade received for 10 students taking some class. The curious thing about this – highly fictitious – data set is that increasing your effort *always* increases your grade. It might be by a lot or it might be by a little, but increasing effort will never decrease your grade. The data are stored in `effort.Rdata`:

```
> load( "effort.Rdata" )
> who(TRUE)
-- Name --      -- Class --      -- Size --
effort         data.frame      10 x 2
$hours         numeric         10
$grade         numeric         10
```

The raw data look like this:

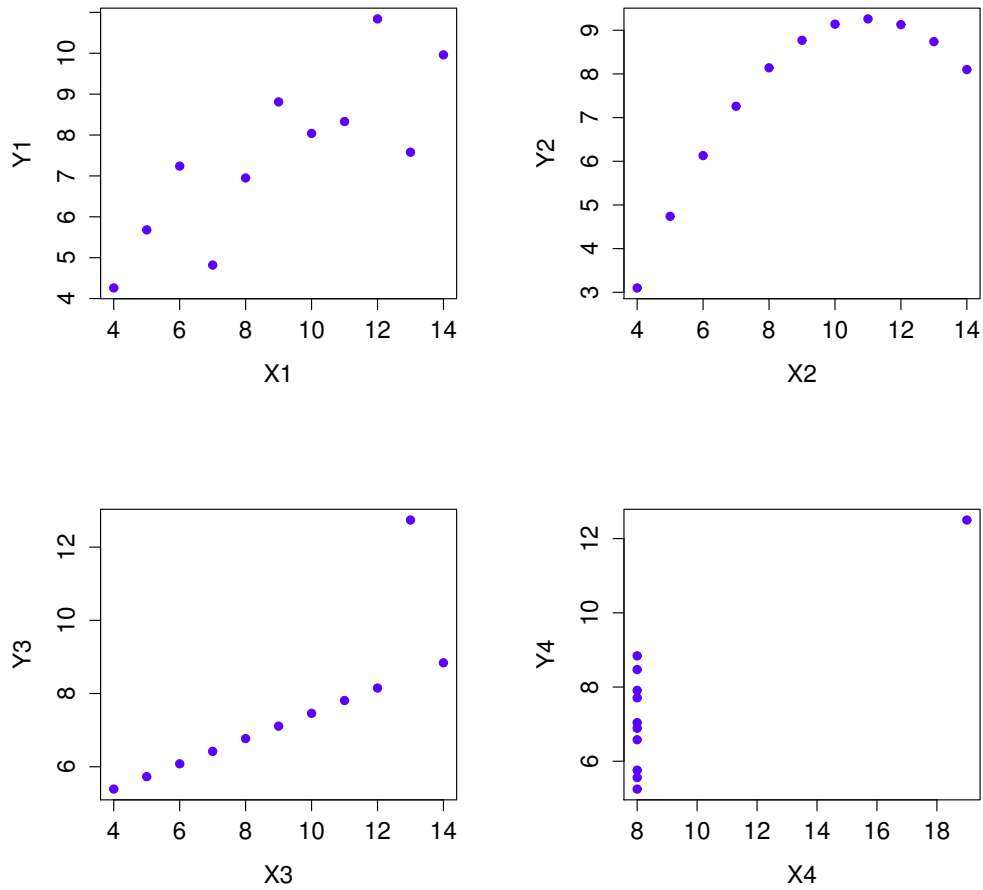


Figure 5.10: Anscombe's quartet. All four of these data sets have a Pearson correlation of $r = .816$, but they are qualitatively different from one another.

```
> effort
  hours grade
1     2    13
2    76    91
3    40    79
4     6    14
5    16    21
6    28    74
7    27    47
8    59    85
9    46    84
10   68    88
```

If we run a standard Pearson correlation, it shows a strong relationship between hours worked and grade received,

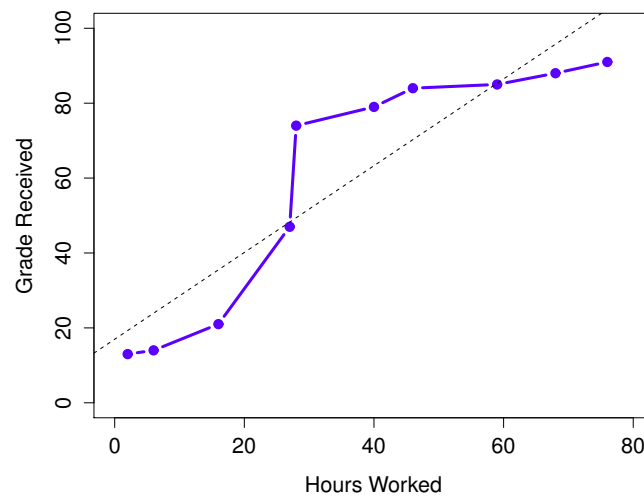


Figure 5.11: The relationship between hours worked and grade received, for a toy data set consisting of only 10 students (each circle corresponds to one student). The dashed line through the middle shows the linear relationship between the two variables. This produces a strong Pearson correlation of $r = .91$. However, the interesting thing to note here is that there’s actually a perfect monotonic relationship between the two variables: in this toy example at least, increasing the hours worked always increases the grade received, as illustrated by the solid line. This is reflected in a Spearman correlation of $\rho = 1$. With such a small data set, however, it’s an open question as to which version better describes the actual relationship involved.

.....

```
> cor( effort$hours, effort$grade )
[1] 0.909402
```

but this doesn’t actually capture the observation that increasing hours worked *always* increases the grade. There’s a sense here in which we want to be able to say that the correlation is *perfect* but for a somewhat different notion of what a “relationship” is. What we’re looking for is something that captures the fact that there is a perfect **ordinal relationship** here. That is, if student 1 works more hours than student 2, then we can guarantee that student 1 will get the better grade. That’s not what a correlation of $r = .91$ says at all.

How should we address this? Actually, it’s really easy: if we’re looking for ordinal relationships, all we have to do is treat the data as if it were ordinal scale! So, instead of measuring effort in terms of “hours worked”, lets rank all 10 of our students in order of hours worked. That is, student 1 did the least work out of anyone (2 hours) so they get the lowest rank (rank = 1). Student 4 was the next laziest, putting in only 6 hours of work in over the whole semester, so they get the next lowest rank (rank = 2). Notice that I’m using “rank =1” to mean “low rank”. Sometimes in everyday language we talk about “rank = 1” to mean “top rank” rather than “bottom rank”. So be careful: you can rank “from smallest value to largest value” (i.e., small equals rank 1) or you can rank “from largest value to smallest value” (i.e., large equals rank 1). In this case, I’m ranking from smallest to largest, because that’s the default way that R does it. But in real life, it’s really easy to forget which way you set things up, so you have to put a bit of effort into remembering!

Okay, so let's have a look at our students when we rank them from worst to best in terms of effort and reward:

	rank (hours worked)	rank (grade received)
student 1	1	1
student 2	10	10
student 3	6	6
student 4	2	2
student 5	3	3
student 6	5	5
student 7	4	4
student 8	8	8
student 9	7	7
student 10	9	9

Hm. These are *identical*. The student who put in the most effort got the best grade, the student with the least effort got the worst grade, etc. We can get R to construct these rankings using the `rank()` function, like this:

```
> hours.rank <- rank( effort$hours ) # rank students by hours worked
> grade.rank <- rank( effort$grade ) # rank students by grade received
```

As the table above shows, these two rankings are identical, so if we now correlate them we get a perfect relationship:

```
> cor( hours.rank, grade.rank )
[1] 1
```

What we've just re-invented is **Spearman's rank order correlation**, usually denoted ρ to distinguish it from the Pearson correlation r . We can calculate Spearman's ρ using R in two different ways. Firstly we could do it the way I just showed, using the `rank()` function to construct the rankings, and then calculate the Pearson correlation on these ranks. However, that's way too much effort to do every time. It's much easier to just specify the `method` argument of the `cor()` function.

```
> cor( effort$hours, effort$grade, method = "spearman")
[1] 1
```

The default value of the `method` argument is `"pearson"`, which is why we didn't have to specify it earlier on when we were doing Pearson correlations.

5.7.7 The `correlate()` function

As we've seen, the `cor()` function works pretty well, and handles many of the situations that you might be interested in. One thing that many beginners find frustrating, however, is the fact that it's not built to handle non-numeric variables. From a statistical perspective, this is perfectly sensible: Pearson and Spearman correlations are only designed to work for numeric variables, so the `cor()` function spits out an error.

Here's what I mean. Suppose you were keeping track of how many `hours` you worked in any given day, and counted how many `tasks` you completed. If you were doing the tasks for money, you might also want to keep track of how much `pay` you got for each job. It would also be sensible to keep track of the

`weekday` on which you actually did the work: most of us don't work as much on Saturdays or Sundays. If you did this for 7 weeks, you might end up with a data set that looks like this one:

```
> load("work.Rdata")

> who(TRUE)
-- Name -- -- Class -- -- Size --
work      data.frame 49 x 7
$hours    numeric    49
$tasks    numeric    49
$pay      numeric    49
$day      integer    49
$weekday  factor      49
$week     numeric    49
$day.type factor      49

> head(work)
  hours tasks pay day weekday week day.type
1  7.2   14  41  1  Tuesday    1 weekday
2  7.4   11  39  2 Wednesday    1 weekday
3  6.6   14  13  3 Thursday    1 weekday
4  6.5   22  47  4  Friday     1 weekday
5  3.1    5   4  5 Saturday    1 weekend
6  3.0    7  12  6  Sunday     1 weekend
```

Obviously, I'd like to know something about how all these variables correlate with one another. I could correlate `hours` with `pay` quite using `cor()`, like so:

```
> cor(work$hours,work$pay)
[1] 0.7604283
```

But what if I wanted a quick and easy way to calculate all pairwise correlations between the numeric variables? I can't just input the `work` data frame, because it contains two factor variables, `weekday` and `day.type`. If I try this, I get an error:

```
> cor(work)
Error in cor(work) : 'x' must be numeric
```

In order to get the correlations that I want using the `cor()` function, I create a new data frame that doesn't contain the factor variables, and then feed that new data frame into the `cor()` function. It's not actually very hard to do that, and I'll talk about how to do it properly in Section 7.5. But it would be nice to have some function that is smart enough to just ignore the factor variables. That's where the `correlate()` function in the `lsr` package can be handy. If you feed it a data frame that contains factors, it knows to ignore them, and returns the pairwise correlations only between the numeric variables:

```
> correlate(work)

CORRELATIONS
=====
- correlation type: pearson
- correlations shown only when both variables are numeric

      hours  tasks  pay  day weekday  week day.type
hours      .  0.800 0.760 -0.049      .  0.018      .
tasks  0.800      .  0.720 -0.072      . -0.013      .
```

```

pay      0.760 0.720 . 0.137 . 0.196 .
day     -0.049 -0.072 0.137 . . 0.990 .
weekday . . . . . . .
week     0.018 -0.013 0.196 0.990 . . .
day.type . . . . . . .

```

The output here shows a `.` whenever one of the variables is non-numeric. It also shows a `.` whenever a variable is correlated with itself (it's not a meaningful thing to do). The `correlate()` function can also do Spearman correlations, by specifying the `corr.method` to use:

```

> correlate( work, corr.method="spearman" )

CORRELATIONS
=====
- correlation type:  spearman
- correlations shown only when both variables are numeric

      hours  tasks  pay   day weekday  week day.type
hours      .  0.805 0.745 -0.047 . 0.010 .
tasks     0.805 .  0.730 -0.068 . -0.008 .
pay       0.745 0.730 .  0.094 . 0.154 .
day      -0.047 -0.068 0.094 . . 0.990 .
weekday   . . . . . . .
week      0.010 -0.008 0.154 0.990 . . .
day.type  . . . . . . .

```

Obviously, there's no new functionality in the `correlate()` function, and any advanced R user would be perfectly capable of using the `cor()` function to get these numbers out. But if you're not yet comfortable with extracting a subset of a data frame, the `correlate()` function is for you.

5.8 ---

Handling missing values

There's one last topic that I want to discuss briefly in this chapter, and that's the issue of **missing data**. Real data sets very frequently turn out to have missing values: perhaps someone forgot to fill in a particular survey question, for instance. Missing data can be the source of a lot of tricky issues, most of which I'm going to gloss over. However, at a minimum, you need to understand the basics of handling missing data in R.

5.8.1 The single variable case

Let's start with the simplest case, in which you're trying to calculate descriptive statistics for a single variable which has missing data. In R, this means that there will be `NA` values in your data vector. Let's create a variable like that:

```

> partial <- c(10, 20, NA, 30)

```

Let's assume that you want to calculate the mean of this variable. By default, R assumes that you want to calculate the mean using all four elements of this vector, which is probably the safest thing for a dumb

automaton to do, but it's rarely what you actually want. Why not? Well, remember that the basic interpretation of `NA` is "I don't know what this number is". This means that $1 + NA = NA$: if I add 1 to some number that I don't know (i.e., the `NA`) then the answer is *also* a number that I don't know. As a consequence, if you don't explicitly tell R to ignore the `NA` values, and the data set does have missing values, then the output will itself be a missing value. If I try to calculate the mean of the `partial` vector, without doing anything about the missing value, here's what happens:

```
> mean( x = partial )
[1] NA
```

Technically correct, but deeply unhelpful.

To fix this, all of the descriptive statistics functions that I've discussed in this chapter (with the exception of `cor()` which is a special case I'll discuss below) have an optional argument called `na.rm`, which is shorthand for "remove NA values". By default, `na.rm = FALSE`, so R does nothing about the missing data problem. Let's try setting `na.rm = TRUE` and see what happens:

When calculating sums and means when missing data are present (i.e., when there are `NA` values) there's actually an additional argument to the function that you should be aware of. This argument is called `na.rm`, and is a logical value indicating whether R should ignore (or "remove") the missing data for the purposes of doing the calculations. By default, R assumes that you want to keep the missing values, so unless you say otherwise it will set `na.rm = FALSE`. However, R assumes that $1 + NA = NA$: if I add 1 to some number that I don't know (i.e., the `NA`) then the answer is *also* a number that I don't know. As a consequence, if you don't explicitly tell R to ignore the `NA` values, and the data set does have missing values, then the output will itself be a missing value. This is illustrated in the following extract:

```
> mean( x = partial, na.rm = TRUE )
[1] 20
```

Notice that the mean is 20 (i.e., $60 / 3$) and *not* 15. When R ignores a `NA` value, it genuinely ignores it. In effect, the calculation above is identical to what you'd get if you asked for the mean of the three-element vector `c(10, 20, 30)`.

As indicated above, this isn't unique to the `mean()` function. Pretty much all of the other functions that I've talked about in this chapter have an `na.rm` argument that indicates whether it should ignore missing values. However, its behaviour is the same for all these functions, so I won't waste everyone's time by demonstrating it separately for each one.

5.8.2 Missing values in pairwise calculations

I mentioned earlier that the `cor()` function is a special case. It doesn't have an `na.rm` argument, because the story becomes a lot more complicated when more than one variable is involved. What it does have is an argument called `use` which does roughly the same thing, but you need to think little more carefully about what you want this time. To illustrate the issues, let's open up a data set that has missing values, `parenthood2.Rdata`. This file contains the same data as the original `parenthood` data, but with some values deleted. It contains a single data frame, `parenthood2`:

```
> load( "parenthood2.Rdata" )
> print( parenthood2 )
  dan.sleep baby.sleep dan.grump day
1      7.59         NA       56   1
2      7.91      11.66       60   2
3      5.14       7.92       82   3
4      7.71       9.61       55   4
```

```

5      6.68      9.75      NA    5
6      5.99      5.04      72    6
BLAH BLAH BLAH

```

If I calculate my descriptive statistics using the `describe()` function

```

> describe( parenthood2 )
      var    n mean    sd median trimmed   mad   min    max    BLAH
dan.sleep    1  91  6.98  1.02   7.03   7.02  1.13  4.84   9.00   BLAH
baby.sleep    2  89  8.11  2.05   8.20   8.13  2.28  3.25  12.07   BLAH
dan.grump     3  92 63.15  9.85  61.00  62.66 10.38 41.00  89.00   BLAH
day           4 100 50.50 29.01  50.50  50.50 37.06  1.00 100.00   BLAH

```

we can see from the `n` column that there are 9 missing values for `dan.sleep`, 11 missing values for `baby.sleep` and 8 missing values for `dan.grump`.²² Suppose what I would like is a correlation matrix. And let's also suppose that I don't bother to tell R how to handle those missing values. Here's what happens:

```

> cor( parenthood2 )
      dan.sleep baby.sleep dan.grump day
dan.sleep      1      NA      NA    NA
baby.sleep     NA      1      NA    NA
dan.grump      NA      NA      1    NA
day            NA      NA      NA     1

```

Annoying, but it kind of makes sense. If I don't *know* what some of the values of `dan.sleep` and `baby.sleep` actually are, then I can't possibly *know* what the correlation between these two variables is either, since the formula for the correlation coefficient makes use of every single observation in the data set. Once again, it makes sense: it's just not particularly *helpful*.

To make R behave more sensibly in this situation, you need to specify the `use` argument to the `cor()` function. There are several different values that you can specify for this, but the two that we care most about in practice tend to be `"complete.obs"` and `"pairwise.complete.obs"`. If we specify `use = "complete.obs"`, R will completely ignore all cases (i.e., all rows in our `parenthood2` data frame) that have any missing values at all. So, for instance, if you look back at the extract earlier when I used the `head()` function, notice that observation 1 (i.e., day 1) of the `parenthood2` data set is missing the value for `baby.sleep`, but is otherwise complete? Well, if you choose `use = "complete.obs"` R will ignore that row completely: that is, even when it's trying to calculate the correlation between `dan.sleep` and `dan.grump`, observation 1 will be ignored, because the value of `baby.sleep` is missing for that observation. Here's what we get:

```

> cor(parenthood2, use = "complete.obs")
      dan.sleep baby.sleep dan.grump    day
dan.sleep  1.0000000  0.6394985 -0.89951468  0.06132891
baby.sleep  0.6394985  1.0000000 -0.58656066  0.14555814
dan.grump  -0.89951468 -0.5865607  1.00000000 -0.06816586
day         0.06132891  0.1455581 -0.06816586  1.00000000

```

The other possibility that we care about, and the one that tends to get used more often in practice, is to set `use = "pairwise.complete.obs"`. When we do that, R only looks at the variables that it's trying to correlate when determining what to drop. So, for instance, since the only missing value for observation 1 of `parenthood2` is for `baby.sleep` R will only drop observation 1 when `baby.sleep` is one of the variables involved: and so R keeps observation 1 when trying to correlate `dan.sleep` and `dan.grump`. When we do it this way, here's what we get:

²²It's worth noting that, even though we have missing data for each of these variables, the output doesn't contain any NA values. This is because, while `describe()` also has an `na.rm` argument, the default value for this function is `na.rm = TRUE`.

```
> cor(parenthood2, use = "pairwise.complete.obs")
      dan.sleep baby.sleep dan.grump day
dan.sleep 1.00000000 0.61472303 -0.903442442 -0.076796665
baby.sleep 0.61472303 1.00000000 -0.567802669 0.058309485
dan.grump -0.90344244 -0.56780267 1.000000000 0.005833399
day        -0.07679667 0.05830949 0.005833399 1.000000000
```

Similar, but not quite the same. It's also worth noting that the `correlate()` function (in the `lsr` package) automatically uses the “pairwise complete” method:

```
> correlate(parenthood2)

CORRELATIONS
=====
- correlation type: pearson
- correlations shown only when both variables are numeric

      dan.sleep baby.sleep dan.grump day
dan.sleep      .      0.615   -0.903 -0.077
baby.sleep     0.615      .   -0.568 0.058
dan.grump     -0.903   -0.568      . 0.006
day           -0.077    0.058    0.006      .
```

The two approaches have different strengths and weaknesses. The “pairwise complete” approach has the advantage that it keeps more observations, so you’re making use of more of your data and (as we’ll discuss in tedious detail in Chapter 10) and it improves the reliability of your estimated correlation. On the other hand, it means that every correlation in your correlation matrix is being computed from a slightly different set of observations, which can be awkward when you want to compare the different correlations that you’ve got.

So which method should you use? It depends a lot on *why* you think your values are missing, and probably depends a little on how paranoid you are. For instance, if you think that the missing values were “chosen” completely randomly²³ then you’ll probably want to use the pairwise method. If you think that missing data are a cue to thinking that the whole observation might be rubbish (e.g., someone just selecting arbitrary responses in your questionnaire), but that there’s no pattern to which observations are “rubbish” then it’s probably safer to keep only those observations that are complete. If you think there’s something systematic going on, in that some observations are more likely to be missing than others, then you have a much trickier problem to solve, and one that is beyond the scope of this book.

5.9

Summary

Calculating some basic descriptive statistics is one of the very first things you do when analysing real data, and descriptive statistics are much simpler to understand than inferential statistics, so like every other statistics textbook I’ve started with descriptives. In this chapter, we talked about the following topics:

- *Measures of central tendency.* Broadly speaking, central tendency measures tell you where the data are. There’s three measures that are typically reported in the literature: the mean, median and mode. (Section 5.1)

²³The technical term here is “missing completely at random” (often written MCAR for short). Makes sense, I suppose, but it does sound ungrammatical to me.

- *Measures of variability.* In contrast, measures of variability tell you about how “spread out” the data are. The key measures are: range, standard deviation, interquartile range (Section 5.2)
- *Getting summaries of variables in R.* Since this book focuses on doing data analysis in R, we spent a bit of time talking about how descriptive statistics are computed in R. (Section 5.4 and 5.5)
- *Standard scores.* The z -score is a slightly unusual beast. It’s not quite a descriptive statistic, and not quite an inference. We talked about it in Section 5.6. Make sure you understand that section: it’ll come up again later.
- *Correlations.* Want to know how strong the relationship is between two variables? Calculate a correlation. (Section 5.7)
- *Missing data.* Dealing with missing data is one of those frustrating things that data analysts really wish they didn’t have to think about. In real life it can be hard to do well. For the purpose of this book, we only touched on the basics in Section 5.8

In the next section we’ll move on to a discussion of how to draw pictures! Everyone loves a pretty picture, right? But before we do, I want to end on an important point. A traditional first course in statistics spends only a small proportion of the class on descriptive statistics, maybe one or two lectures at most. The vast majority of the lecturer’s time is spent on inferential statistics, because that’s where all the hard stuff is. That makes sense, but it hides the practical everyday importance of choosing good descriptives. With that in mind...

5.9.1 Epilogue: Good descriptive statistics are descriptive!

The death of one man is a tragedy.

The death of millions is a statistic.

– Josef Stalin, Potsdam 1945

950,000 – 1,200,000

– Estimate of Soviet repression deaths,
1937-1938 (Ellman, 2002)

Stalin’s infamous quote about the statistical character death of millions is worth giving some thought. The clear intent of his statement is that the death of an individual touches us personally and its force cannot be denied, but that the deaths of a multitude are incomprehensible, and as a consequence mere statistics, more easily ignored. I’d argue that Stalin was half right. A statistic is an abstraction, a description of events beyond our personal experience, and so hard to visualise. Few if any of us can imagine what the deaths of millions is “really” like, but we can imagine one death, and this gives the lone death its feeling of immediate tragedy, a feeling that is missing from Ellman’s cold statistical description.

Yet it is not so simple: without numbers, without counts, without a description of what happened, we have *no chance* of understanding what really happened, no opportunity even to try to summon the missing feeling. And in truth, as I write this, sitting in comfort on a Saturday morning, half a world and a whole lifetime away from the Gulags, when I put the Ellman estimate next to the Stalin quote a dull dread settles in my stomach and a chill settles over me. The Stalinist repression is something truly beyond my experience, but with a combination of statistical data and those recorded personal histories that have come down to us, it is not entirely beyond my comprehension. Because what Ellman’s numbers tell us is this: over a two year period, Stalinist repression wiped out the equivalent of every man, woman and child currently alive in the city where I live. Each one of those deaths had its own story, was its own tragedy, and only some of those are known to us now. Even so, with a few carefully chosen statistics, the scale of the atrocity starts to come into focus.

Thus it is no small thing to say that the first task of the statistician and the scientist is to summarise the data, to find some collection of numbers that can convey to an audience a sense of what has happened. This is the job of descriptive statistics, but it's not a job that can be told solely using the numbers. You are a data analyst, not a statistical software package. Part of your job is to take these *statistics* and turn them into a *description*. When you analyse data, it is not sufficient to list off a collection of numbers. Always remember that what you're really trying to do is communicate with a human audience. The numbers are important, but they need to be put together into a meaningful story that your audience can interpret. That means you need to think about framing. You need to think about context. And you need to think about the individual events that your statistics are summarising.